# SQLite con Room

SQLite es la base de datos que ya trae el teléfono: un archivo con tablas y filas. Room es la librería que escribe el SQL a partir de clases Kotlin. Esta documentación fija qué se va a guardar en el registro y cómo se reparte el trabajo. La guía paso a paso, con el código para pegar, viene después.

Hoy el nombre y el correo del formulario quedan en DataStore, un solo par de textos. Cada vez que alguien pulsa **Registrarse** con los datos válidos, ese par se reemplaza. No queda una lista de las personas que se registraron. La foto sigue siendo un archivo en la caché, y la ficha solo recuerda su dirección.

## Qué va a hacer la app

Al pulsar **Registrarse** con el formulario válido, además de guardar el nombre y el correo en DataStore, la app inserta una fila en SQLite. En la misma pantalla de registro, debajo del formulario, aparece la lista **Registros guardados**. Cada fila muestra el nombre, el correo y la fecha. Un botón en la fila borra ese registro.

La foto sigue siendo opcional. Si hay una, la fila guarda su dirección. Si no hay, la fila se guarda igual.

La ruta `register_page` y el `NavHost` no cambian. La base entra en un paquete `data`, en `RegistroViewModel` y en `RegisterScreen`.

## Room, en tres piezas

La app no escribe `CREATE TABLE` a mano. Describe la tabla y las consultas, y Room genera el SQL.

- La **entidad** es la fila. Una clase `Registro` con las columnas.
- El **DAO** es el acceso. Una interfaz `RegistroDao` con insertar, listar y borrar.
- La **base** es el archivo. Una clase `AppDatabase` que junta la entidad y el DAO, y abre `registro.db`.

`Registro` lleva estas columnas:

| Columna | Tipo | Qué guarda |
|---|---|---|
| `id` | número, clave | Lo asigna SQLite al insertar |
| `nombre` | texto | El nombre del formulario en ese envío |
| `correo` | texto | El correo de ese envío |
| `fotoUri` | texto | La dirección de la foto, o vacío |
| `creadoEn` | número | El instante del envío, en milisegundos |

El mismo correo puede aparecer en varias filas. Cada envío es un registro distinto. `aceptaTerminos` no es columna: la fila solo se inserta cuando la validación ya exigió la casilla

marcada.

La base queda en la versión 1. `exportSchema = false` deja el esquema en las anotaciones, sin un archivo JSON de historial.

## DataStore y Room

Los dos siguen en la app. Guardan cosas distintas.

| Dato | Dónde | Para qué |
|---|---|---|
| Borrador del nombre y del correo | DataStore | Rellenar el formulario al volver a abrir la app |
| Cada registro enviado | SQLite, tabla `registros` | La lista de personas registradas |
| Archivo de la foto | Caché | La imagen de la toma actual |
| Dirección de la foto en la ficha | `RegistroViewModel` | Mostrarla mientras la pantalla sigue viva |
| Dirección de la foto en la fila | Columna `fotoUri` | Recordar qué toma iba con ese envío |

`RegistroPreferences` no se elimina. Sigue leyendo y escribiendo el borrador. Room no reemplaza ese archivo de preferencias.

La caché de la foto puede borrarse al cerrar la app. La fila conserva el texto de la dirección. Si el archivo ya no está, la lista igual muestra el nombre y el correo.

## Dependencias

Room lee las anotaciones y genera el código del DAO. Ese trabajo lo hace KSP, el procesador de símbolos de Kotlin. Hay que declarar el plugin y tres librerías. Hoy el proyecto no las tiene.

En `app/build.gradle.kts`, el bloque `plugins` queda así:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    id("com.google.devtools.ksp") version "2.2.10-2.0.2"
}
```

`2.2.10-2.0.2` corresponde al Kotlin `2.2.10` del proyecto. El número de KSP tiene que seguir a esa versión de Kotlin.

Dentro de `dependencies`:

```
implementation("androidx.room:room-runtime:2.8.5")
implementation("androidx.room:room-ktx:2.8.5")
ksp("androidx.room:room-compiler:2.8.5")
```

- `room-runtime` es Room en la app: abre el archivo SQLite y ejecuta las consultas.
- `room-ktx` permite que el DAO use funciones suspendidas y `Flow`.
- `room-compiler`, con `ksp`, genera la implementación de `RegistroDao` y de `AppDatabase` al compilar.

Después de pegarlas hay que sincronizar Gradle.

## Qué cambia en el proyecto

| Pieza | Dónde | Qué aporta |
|---|---|---|
| Plugin KSP y librerías Room | `app/build.gradle.kts` | Generan el SQL al compilar |
| Fila | `data/Registro.kt` | La entidad y sus columnas |
| Consultas | `data/RegistroDao.kt` | Insertar, observar la lista y borrar |
| Archivo de la base | `data/AppDatabase.kt` | Abre `registro.db` una vez por proceso |
| Envío y lista | `RegistroViewModel` | Inserta al guardar y publica las filas |
| Lista en pantalla | `RegisterScreen` | Dibuja los registros y el botón de borrar |

`RegistroPreferences` y la cámara no se reescriben. El insert usa el nombre, el correo y `fotoUri` que la ficha ya tiene en el momento de pulsar **Registrarse**.

## Recorrido al pulsar Registrarse

1. `validar()` revisa nombre, correo y términos, como ahora.
2. Si hay errores, no se escribe ninguna fila.
3. Si el formulario es válido, `guardar()` escribe el borrador en DataStore y inserta una fila con el nombre, el correo, la dirección de la foto y la hora.
4. El DAO expone la tabla como un `Flow`. Al insertar, ese flujo emite la lista nueva.
5. La pantalla, que observa el flujo a través del `ViewModel`, dibuja la fila.

Al pulsar borrar en una fila, el DAO elimina ese `id`. El flujo vuelve a emitir y la fila desaparece de la lista. El borrador de DataStore y el archivo de la foto no se tocan.

La consulta no corre en el hilo que dibuja la pantalla. Las funciones del DAO son suspendidas, y el `ViewModel` las lanza en `viewModelScope`.

## Qué queda fuera de esta etapa

- Editar una fila ya guardada.
- Impedir correos repetidos.

- Copiar el JPEG a una carpeta permanente para que la foto sobreviva al cierre de la app.
- Una pantalla nueva o un cambio de ruta.
- Migraciones: la base nace en la versión 1.
- El archivo JSON de esquema (`exportSchema` queda en `false`).

## Qué va a traer la guía

La guía de implementación está en `room.md`. Arma el plugin, las tres clases de Room, el insert y el borrado en el `ViewModel`, y la lista en `RegisterScreen`.